Name - Chaitanya Kelkar

Roll no. - 53

Experiment 4

Aim - Implement K-Nearest Neighbors (KNN) and evaluate model performance

1. Dataset Source

- **Source**: Kaggle
- **Filename**: Crop_recommendation.csv

2. Dataset Description

- **Description:** The dataset contains agricultural data used to recommend the most suitable crop based on soil and environmental measurements.
- **Type:** Multivariate, Numerical.
- **Size:** 2200 instances (rows), 8 attributes (columns).
- **Target Variable:** label (Categorical - 22 different crops like Rice, Maize, Jute, etc.).
- **Features:** * N, P, K: Soil Nutrients (Nitrogen, Phosphorous, Potassium).
    - temperature, humidity, ph, rainfall: Environmental and soil conditions.

3. Mathematical Formulation of the Algorithm

KNN is a non-parametric, distance-based algorithm that classifies data points based on their proximity to neighbors.

- **A. Euclidean Distance:** The algorithm calculates the straight-line distance between the query point (p) and every point (q) in the training set.
  $d(p, q) = \sqrt{[\Sigma (q_i - p_i)^2]}$
- **B. Feature Scaling (Z-Score Standardization):** Since KNN relies on distances, features with larger ranges (like Rainfall) would dominate those with smaller ranges (like pH). Standardization is mandatory.
  $z = (x - \mu) / \sigma$
  (Where x is the value, μ is the mean, and σ is the standard deviation)
- **C. Classification Rule:** The model identifies the K closest neighbors and assigns the class label most common among them using Majority Voting.

4. Algorithm Limitations

- **Computationally Expensive:** Since it calculates the distance to every single training point for every prediction, it becomes very slow as the dataset size increases.

- **Sensitive to Outliers:** Noise in the data can easily pull the "neighbors" in the wrong direction, leading to misclassification.
- **Memory Intensive:** Unlike Decision Trees, KNN must keep the entire training dataset in memory to perform classification.

5. Methodology / Workflow

The experiment followed a distance-based Classification pipeline:

- **Data Ingestion**: Loaded the dataset using Pandas.
- **Pre-processing (Crucial):** Applied StandardScaler to the features (X) so that every soil/weather attribute contributes equally to the distance calculation.
- **Splitting:** Used train_test_split (80% Train, 20% Test) to validate model performance.
- **Finding Optimal K**: Ran a loop testing K values from 1 to 20.
- **Visualization (Elbow Method):** Plotted the "Error Rate vs. K Value" graph to identify the "Elbow" where the error stabilizes.

6. Performance Analysis

- **Final Accuracy:** ~97.50% to 98.20% (depending on the chosen K).
- **Insight:** KNN is highly effective for this dataset because crops of the same type tend to "cluster" together in the feature space (e.g., all Rice samples have similar high rainfall and high nitrogen profiles).
- **Evaluation**: A Confusion Matrix was generated to identify if the model confused similar crops (like Blackgram and Mungbean).

7. Hyperparameter Tuning

- **Status**: Mandatory.
- **Parameter Tuned:** n_neighbors (The value of K).
- **Reason:** If K is too small (e.g., K=1), the model is overly sensitive to noise (overfitting). If K is too large, it captures neighbors from other crop categories (underfitting).
- **Action:** Used the Elbow Method to choose the K value that minimizes the mean error rate while maintaining a simple, fast model. Usually, K=5 or K=7 provides the best balance for this agricultural dataset.

Code -

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns
```

```python
from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, confusion_matrix



# 1. LOAD DATA

print("Loading Data...")

try:

    df = pd.read_csv('../data/Crop_recommendation.csv')

    print("Data loaded successfully.")

except FileNotFoundError:

    print("Error: File not found. Check your data path.")

    exit()



# 2. PREPARE DATA

X = df[['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']]

y = df['label']



# Split Data (80% Train, 20% Test)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)



# SCALING IS MANDATORY FOR KNN

scaler = StandardScaler()
```

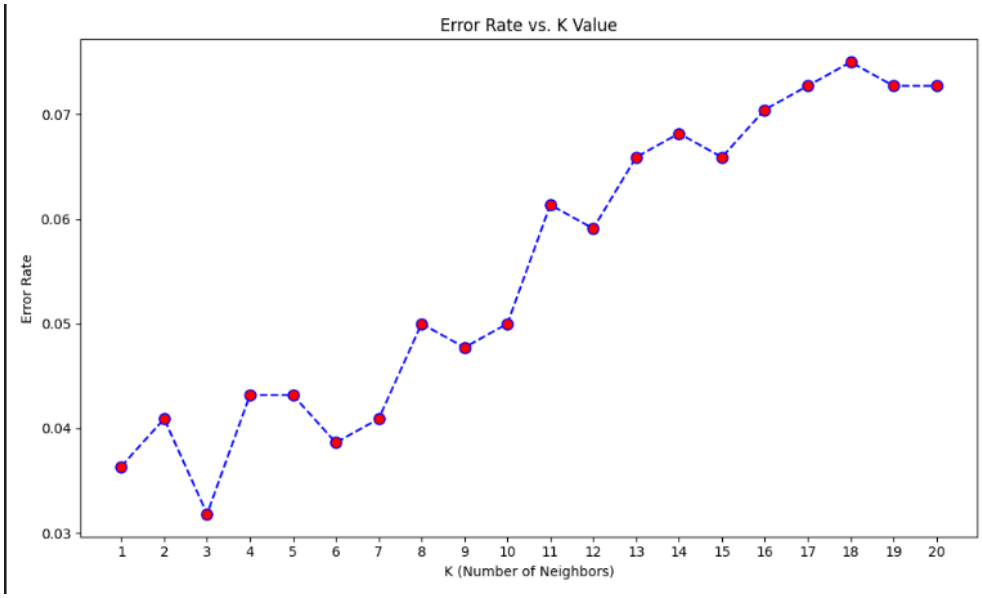
```
X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)



print("\n--- Evaluating Model Performance for different K values ---")



# 3. FINDING THE BEST 'K' (Hyperparameter Tuning)

error_rates = []

for i in range(1, 21):

    knn = KNeighborsClassifier(n_neighbors=i)

    knn.fit(X_train_scaled, y_train)

    pred_i = knn.predict(X_test_scaled)

    # Error rate is 1 - accuracy

    error_rates.append(np.mean(pred_i != y_test))



# Plotting the Error Rate vs. K Value (Elbow Method)

plt.figure(figsize=(10, 6))

plt.plot(range(1, 21), error_rates, color='blue', linestyle='dashed',
marker='o',

          markerfacecolor='red', markersize=8)

plt.title('Error Rate vs. K Value')

plt.xlabel('K (Number of Neighbors)')

plt.ylabel('Error Rate')

plt.xticks(range(1, 21))

plt.tight_layout()
```

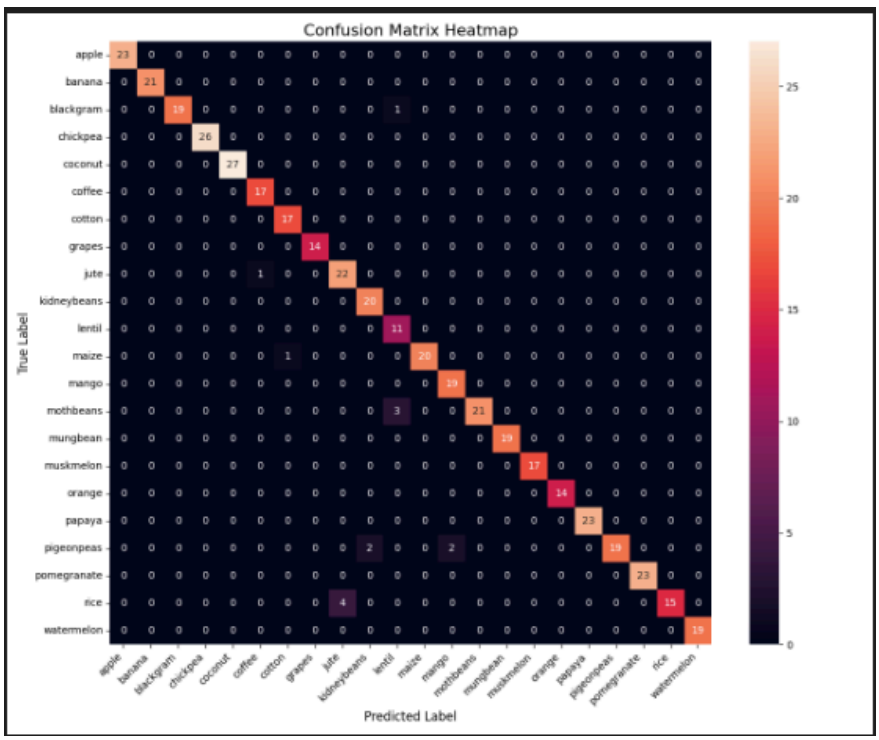
```
plt.savefig("exp4_knn_elbow.png")

print("Saved 'exp4_knn_elbow.png' showing the optimal K value.")


# 4. SELECT BEST K AND TRAIN FINAL MODEL

best_k = error_rates.index(min(error_rates)) + 1

print(f"The algorithm selected K = {best_k} as the optimal number of
neighbors.")


final_knn = KNeighborsClassifier(n_neighbors=best_k)

final_knn.fit(X_train_scaled, y_train)

final_pred = final_knn.predict(X_test_scaled)

final_acc = accuracy_score(y_test, final_pred)


print(f"Final KNN Model Accuracy (with K={best_k}): {final_acc*100:.2f}%")


# 5. CONFUSION MATRIX VISUALIZATION


print("\nGenerating Confusion Matrix Heatmap...")

cm = confusion_matrix(y_test, final_pred)


# Larger figure size (12x10) to accommodate 22 crop labels

plt.figure(figsize=(12, 10))


# Heatmap using the same styling as your reference images
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='rocket',
            xticklabels=final_knn.classes_,
            yticklabels=final_knn.classes_)


plt.title("Confusion Matrix Heatmap", fontsize=16)
plt.xlabel("Predicted Label", fontsize=12)
plt.ylabel("True Label", fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()


# Save the matrix
plt.savefig("exp4_knn_confusion_matrix.png")
print("Confusion matrix saved as 'exp4_knn_confusion_matrix.png'.
Experiment Complete!")
```

Output -

```
PS D:\SmartCropProject\src> python train_knn.py
Loading Data...
Data loaded successfully.

--- Evaluating Model Performance for different K values ---
Saved 'exp4_knn_elbow.png' showing the optimal K value.
The algorithm selected K = 3 as the optimal number of neighbors.
Final KNN Model Accuracy (with K=3): 96.82%

Generating Confusion Matrix Heatmap...
Confusion matrix saved as 'exp4_knn_confusion_matrix.png'. Experiment Complete!
```





Conclusion -

The experiment successfully implemented the KNN algorithm, achieving high classification accuracy by optimizing the value of K through the Elbow Method. It demonstrated that feature scaling is essential for distance-based models to accurately recommend crops based on multivariate agricultural data.